Partial-Binding Witness Encryption over Groth16

Stephen Duan
stephen.d@zkm.io

2026-04-19

Abstract

BABE's witness encryption for Groth16 binds the ciphertext to the full public-input vector at encryption time, precluding protocols where some inputs are determined after encryption. We introduce *partial-binding WE*, which binds only a designated static prefix x_S and lets the remaining indices x_D be committed at dispute time via a *Dual-Scalar Garbled Circuit* (DSGC) that outputs both $r \cdot \pi_1$ and a blinded dynamic term $r \cdot P_D + r \cdot B$, where $P_D = \sum_{j \in D} x_j L_j$ and $B \in \mathbb{G}_1$ is a verifier-chosen blinding point. The DSGC structure binds x_D proactively to the specific proof used in decryption while preventing low-dimensional leakage of $\{r \cdot L_j\}_{j \in D}$. Overhead is $|D|$ additional fixed-base scalar multiplications inside the garbled circuit plus one blinded offset. Security reduces to basic BABE plus GC-security and Lamport one-wayness. The scheme retains BABE's property of requiring no on-chain Groth16 verifier.

1 Introduction

Witness encryption (WE) over Groth16 [2], as instantiated by BABE [1], produces a ciphertext that decrypts to a hidden message iff the decryptor supplies a valid Groth16 proof π for a fixed statement (vk, x). The encryption procedure $\mathsf{Enc}(vk, x, \text{msg})$ consumes the entire public-input vector x at setup time.

This rigidity is a deployment obstacle. Consider a protocol in which a Verifier encrypts at time T_0 and a Prover decrypts at time $T_1 > T_0$, where some of the public-input values are determined by events occurring in $[T_0, T_1]$ (e.g. blockchain state evolving between setup and proof time). At T_0 the Verifier has access only to a prefix x_S of x; the suffix x_D is revealed at T_1. Basic BABE cannot encrypt under these constraints.

1.1 Contribution

We present partial-binding WE, a modification of BABE in which Enc takes only $(vk, x_S, S, D, \text{msg})$ as input and produces an augmented ciphertext. Decryption succeeds for any valid (x_S, x_D, π) with x_S matching the committed prefix. The modification is:

- **Encryption:** replace $Y = e(\alpha, \beta) \cdot e(vk_x, \gamma)$ with $Y_S = e(\alpha, \beta) \cdot e(P_S, \gamma)$ where $P_S = \sum_{j \in S} x_j L_j$; sample a blinding point $B \in \mathbb{G}_1$ and fold $e(r \cdot B, \gamma)^{-1}$ into the mask used for ct_3.
- **Decryption:** use the DSGC to compute $(r \cdot \pi_1,\ r \cdot P_D + r \cdot B)$ from (π_1, x_D), strip the dynamic factor and the blinding factor from $r \cdot Y$ via bilinearity, then unmask under the blinded Y_S term.

Security reduces to BABE WE security plus DLog hardness in $\mathbb{G}_1$ (Lemma 2). Decryption remains off-chain, no new Bitcoin-script primitive is introduced, and the on-chain delta is limited to additional Lamport commitments for x_D plus the larger dispute-path GC witness induced by the DSGC.

The construction exploits the linearity of Groth16's public-input aggregation $vk_x = \sum_j x_j \cdot L_j$. SNARKs with non-linear public-input handling do not directly admit this technique.

1.2 Outline

§2 establishes notation and recalls basic BABE. §3 presents our construction. §5 gives the security analysis. §6 summarizes cost.

2 Preliminaries

2.1 Notation

Let $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ denote a Type-III pairing on BN254. The Groth16 verifying key is $vk = (\alpha, \beta, \gamma, [\delta]_2, \{L_j\}_{j=0}^n)$ where $\alpha, L_j \in \mathbb{G}_1$ and $\beta, \gamma, [\delta]_2 \in \mathbb{G}_2$. Equivalently, Groth16 setup samples a scalar $\delta \in \mathbb{F}_r$ and places its $\mathbb{G}_2$ encoding $[\delta]_2$ in the verifying key. A proof is $\pi = (\pi_1, \pi_2, \pi_3) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1$. The public-input vector is $x = (x_0, x_1, \ldots, x_n) \in \mathbb{F}_r^{n+1}$ with $x_0 = 1$. The verifier accepts iff

$$e(\pi_1, \pi_2) \;=\; e(\alpha, \beta) \cdot e(vk_x, \gamma) \cdot e(\pi_3, [\delta]_2), \qquad vk_x \;:=\; \sum_{j=0}^{n} x_j \cdot L_j. \tag{1}$$

$H : \{0,1\}^* \to \{0,1\}^{256}$ is a collision-resistant hash. We write $\mathrm{Adv}_{\mathrm{scheme}}(\lambda)$ for the advantage of a PPT adversary at security parameter λ, and $\mathrm{negl}(\lambda)$ for negligible functions.

2.2 Witness encryption for Groth16

Definition 1 (WE for Groth16)**.** *A witness encryption scheme for Groth16 relation $\mathcal{R}_{vk} = \{(x, w) : \exists \pi = \mathsf{Prove}(vk, x, w)$ accepts$\}$ is a pair of PPT algorithms $(\mathsf{Enc}, \mathsf{Dec})$ such that:*

- Correctness: *for every $(x, w) \in \mathcal{R}_{vk}$,* $\mathsf{Dec}(\mathsf{Prove}(vk, x, w), \mathsf{Enc}(vk, x, \mathrm{msg})) = \mathrm{msg}$.
- Soundness: *for every $x \notin \mathcal{L}_{vk}$ (i.e., $\{w : (x, w) \in \mathcal{R}_{vk}\} = \emptyset$), and every* $\mathrm{msg} \neq \mathrm{msg}'$, $\{\mathsf{Enc}(vk, x, \mathrm{msg})\} \approx_c \{\mathsf{Enc}(vk, x, \mathrm{msg}')\}$.

BABE [1] instantiates $(\mathsf{Enc}, \mathsf{Dec})$ with a pairing-based construction; we recall it below.

2.3 BABE ciphertext structure

A BABE ciphertext per cut-and-choose instance is $ct = (gc\_ct, adaptor, ct_2, ct_3)$ where gc_ct is a garbled circuit computing $r \cdot \pi_1$, *adaptor* binds its output labels to Bitcoin pre-signatures, $ct_2 = r \cdot [\delta]_2$, and $ct_3 = \mathrm{msg} \oplus H(r \cdot Y)$ with $Y = e(\alpha, \beta) \cdot e(vk_x, \gamma)$. The per-instance scalar r and mask msg are derived from a seed via a PRG. Cut-and-choose over N_{CC} instances detects malformed garbling with probability exponential in the opened subset size.

Algorithm 1 BABE $\mathsf{Enc}(vk, x;\, I)$ — Verifier, off-chain, T_0

Input: vk; full public inputs $x = (x_0, \ldots, x_n)$; instance $I = (r, \mathrm{msg}, \Delta, gc\_ct, adaptor)$
Output: $ct = (gc\_ct, adaptor, ct_2, ct_3)$

1: $ct_2 \leftarrow r \cdot [\delta]_2$
2: $vk_x \leftarrow \sum_{j=0}^{n} x_j \cdot L_j$ ▷ requires every x_j
3: $Y \leftarrow e(\alpha, \beta) \cdot e(vk_x, \gamma)$
4: $ct_3 \leftarrow \mathrm{msg} \oplus H(r \cdot Y)$
5: **return** $(gc\_ct, adaptor, ct_2, ct_3)$

Algorithm 2 BABE $\mathsf{Dec}(\pi, ct)$ — Prover, off-chain, T_1

Input: proof $\pi = (\pi_1, \pi_2, \pi_3)$ for (vk, x); ct
Output: msg

1: $ct_1 \leftarrow r \cdot \pi_1$ ▷ via GC unlock; r not revealed
2: $r \cdot Y \leftarrow e(ct_1, \pi_2)/e(\pi_3, ct_2)$ ▷ by (1)
3: $\mathrm{msg} \leftarrow ct_3 \oplus H(r \cdot Y)$
4: **return** msg

Remark 1 (On-chain surface)**.** *Algorithms 1 and 2 run off-chain. Bitcoin script is used only for (i) hash commitments to (ct_2, ct_3) published during C&C, (ii) gate-by-gate GC unlocking at dispute time, (iii) a hashlock on* msg *gating the final UTXO spend. No pairings or target-group arithmetic are evaluated on chain.*

3 Partial-binding witness encryption

3.1 Problem statement

Definition 2 (Partial-binding WE)**.** *Fix a partition $\{0, 1, \ldots, n\} = S \sqcup D$ at circuit compile time with $0 \in S$. A partial-binding WE scheme for $\mathcal{R}_{vk}$ with respect to (S, D) is a pair $(\mathsf{Enc}^\star, \mathsf{Dec}^\star)$ such that:*

- Prefix-only encryption: $\mathsf{Enc}^\star$ *takes input $(vk, x_S, S, D, \mathrm{msg})$; it does not depend on x_D.*
- Correctness: *for every $((x_S, x_D), w) \in \mathcal{R}_{vk}$ with $\pi = \mathsf{Prove}(vk, (x_S, x_D), w)$, it holds that*

$$\mathsf{Dec}^\star\big(\pi,\ (x_S, x_D),\ \mathsf{Enc}^\star(vk, x_S, S, D, \mathrm{msg})\big) = \mathrm{msg}.$$

- Prefix soundness: *for every x_S such that no (x_D, w) satisfies $\mathcal{R}_{vk}$, and every* $\mathrm{msg} \neq \mathrm{msg}'$,

$$\{\mathsf{Enc}^\star(vk, x_S, \cdot, \mathrm{msg})\} \ \approx_c\ \{\mathsf{Enc}^\star(vk, x_S, \cdot, \mathrm{msg}')\}.$$

Partial-binding does not constrain x_D: decryption succeeds for any (x_D, π) with π valid for (x_S, x_D). External mechanisms (e.g. Lamport commitment plus Bitcoin-script equality) must pin x_D for enclosing protocols. We formalize this separation in §5.

3.2 Dual-Scalar Garbled Circuit

Basic BABE's garbled circuit computes a single scalar multiplication $r \cdot \pi_1$. We replace it with a *Dual-Scalar Garbled Circuit* (DSGC) that computes two outputs:

$$\mathsf{DSGC}(\pi_1,\ x_D) \ \longrightarrow\ (r \cdot \pi_1,\ r \cdot P_D + r \cdot B),$$

where $P_D = \sum_{j \in D} x_j \cdot L_j$ is the dynamic-prefix contribution, $\{r \cdot L_j\}_{j \in D}$ are baked into the circuit as garbling-time constants, and $B \in \mathbb{G}_1$ is a verifier-chosen blinding point whose scaled image $r \cdot B$ is also baked into the circuit. The Verifier knows both r and the CRS $\{L_j\}$ at T_0.

The DSGC input-label layout reserves Lamport-committed wires for both π_1 (as in basic BABE) and x_D (new). At dispute, the Prover Lamport-reveals both; BABE's soldering step routes those reveals into the DSGC input wires, producing the two scalar-mul outputs via the standard GC-evaluation path.

Why DSGC binds x_D to the proof. The dynamic DSGC output is uniquely determined by the Lamport-committed x_D: it is not the Prover's arbitrary off-chain choice. The bilinear cancellation in decryption (Algorithm 5, line 4) then fails whenever the proof's public input differs from the Lamport-committed x_D, because the two $e(P_D, \gamma)^r$ factors no longer match. The additive blinding term $r \cdot B$ prevents the evaluator from isolating low-dimensional basis elements such as $r \cdot L_j$ from the DSGC output. We formalize this in §5.

Lamport-x_D soldering (required). The binding in Alg. 4 relies on two properties:

1. *GC privacy* hides the scaled bases $\{r \cdot L_j\}_{j \in D}$ baked into the DSGC as garbling-time constants together with the blinded offset $r \cdot B$. The evaluator learns neither individual constants nor intermediate wire values; only the final outputs $(r \cdot \pi_1, r \cdot P_D + r \cdot B)$ for the specific inputs provided.

2. *Lamport-x_D soldering* ties Lamport preimages of x_D bits to the DSGC input-wire labels, so the only x_D that can be fed into the DSGC is the on-chain-committed one. For each bit (j, i) of x_D, the Verifier samples Lamport preimage pair $(s^0_{j,i}, s^1_{j,i})$, publishes $(H(s^0_{j,i}), H(s^1_{j,i}))$, and derives DSGC input labels $L^b_{j,i} = \mathrm{KDF}(s^b_{j,i})$ so that Lamport reveal uniquely determines the GC input label. This mirrors BABE's soldering of Lamport-π_1 to the GC's π_1-wires.

Without property (2), a Prover could evaluate the DSGC with an arbitrary x'_D (using publicly-derivable input labels) and recover msg using a proof for (x_S, x'_D), defeating the binding. Both properties together are necessary for DSGC binding (Lemma 5).

3.3 Construction

Let

$$P_S \ =\ \sum_{j \in S} x_j \cdot L_j, \qquad P_D \ =\ \sum_{j \in D} x_j \cdot L_j, \qquad Y_S \ =\ e(\alpha, \beta) \cdot e(P_S, \gamma). \tag{2}$$

By linearity of vk_x and bilinearity of e:

$$vk_x \ =\ P_S + P_D, \qquad Y \ =\ Y_S \cdot e(P_D, \gamma). \tag{3}$$

Let $B \in \mathbb{G}_1$ be a fresh verifier-chosen blinding point and write

$$\widetilde{Y}_S \ :=\ Y_S \cdot e(B, \gamma)^{-1}. \tag{4}$$

Algorithm 3 $\mathsf{Enc}^\star(vk, x_S, S, D;\, I)$ — Verifier, off-chain, T_0

Input: vk; static values x_S; partition (S, D); instance $I = (r,\ \text{msg},\ \Delta,\ dsgc\_ct,\ adaptor)$ where $dsgc\_ct$ garbles Alg. 4

Output: $ct = (dsgc\_ct,\ adaptor,\ ct_2,\ ct_3)$

1: $ct_2 \leftarrow r \cdot [\delta]_2$
2: $P_S \leftarrow \sum_{j \in S} x_j \cdot L_j$
3: $Y_S \leftarrow e(\alpha, \beta) \cdot e(P_S, \gamma)$
4: choose random $B \in \mathbb{G}_1$ and bake $r \cdot B$ into $dsgc\_ct$
5: $\widetilde{Y}_S \leftarrow Y_S \cdot e(B, \gamma)^{-1}$
6: $ct_3 \leftarrow \text{msg} \oplus H(r \cdot \widetilde{Y}_S)$ ▷ mask uses blinded Y_S
7: **return** $(dsgc\_ct,\ adaptor,\ ct_2,\ ct_3)$

Algorithm 4 DSGC functionality — garbled at T_0, evaluated at T_2

Input: $\pi_1 \in \mathbb{G}_1$ (Lamport-revealed); $x_D \in \mathbb{F}_r^{|D|}$ (Lamport-revealed)
constants (baked at garbling): $r \in \mathbb{F}_r$, $\{r \cdot L_j\}_{j \in D} \subset \mathbb{G}_1$, $r \cdot B \in \mathbb{G}_1$

Output: $(c_1, c_1') = (r \cdot \pi_1,\ r \cdot P_D + r \cdot B) \in \mathbb{G}_1^2$

1: $c_1 \leftarrow r \cdot \pi_1$ ▷ variable-base scalar-mul
2: $c_1' \leftarrow \sum_{j \in D} x_j \cdot (r \cdot L_j) + r \cdot B$ ▷ $|D|$ fixed-base scalar-muls, summed, then blinded
3: **return** (c_1, c_1')

Algorithm 5 $\mathsf{Dec}^\star(\pi, x = (x_S, x_D), ct)$ — Prover, off-chain, T_2

Input: $\pi = (\pi_1, \pi_2, \pi_3)$ for $(vk, (x_S, x_D))$; ct

Output: msg

1: $(ct_1, ct_1') \leftarrow \mathsf{DSGC}(\pi_1,\ x_D)$ ▷ via GC unlock; both outputs together
2: $Q_{\text{blind}} \leftarrow e(ct_1',\ \gamma)$ ▷ $= e(P_D, \gamma)^r \cdot e(B, \gamma)^r$
3: $r \cdot Y \leftarrow e(ct_1,\ \pi_2)/e(\pi_3,\ ct_2)$ ▷ by (1)
4: $r \cdot \widetilde{Y}_S \leftarrow (r \cdot Y)/Q_{\text{blind}}$ ▷ by (3) and (4)
5: $\text{msg} \leftarrow ct_3 \oplus H(r \cdot \widetilde{Y}_S)$
6: **return** msg

Lemma 1 (Correctness)**.** *Algorithms 3 and 5 satisfy the correctness condition of Definition 2, provided the DSGC is garbled consistently with Algorithm 4 and the Prover supplies Lamport-revealed (π_1, x_D) matching the proof's public input.*

Proof. Let π be valid for $(vk, (x_S, x_D))$. The DSGC on (π_1, x_D) outputs $ct_1 = r \cdot \pi_1$ and $ct_1' = r \cdot P_D + r \cdot B$. By (1), $e(ct_1, \pi_2)/e(\pi_3, ct_2) = r \cdot Y$. Line 2 yields $Q_{\text{blind}} = e(P_D, \gamma)^r \cdot e(B, \gamma)^r$. Applying (3) and (4): $r \cdot Y / Q_{\text{blind}} = r \cdot \widetilde{Y}_S$. Since $ct_3 = \text{msg} \oplus H(r \cdot \widetilde{Y}_S)$, line 5 recovers msg. □

3.4 On-chain surface

Algorithms 3–5 run entirely off-chain. The on-chain surface of partial-binding equals that of basic BABE plus Lamport commitments on x_D in the Assert transaction (routed into DSGC input wires, analogous to Lamport-on-π_1). No new Bitcoin-script primitive is required; the deltas are additional Lamport slots per bit of x_D and a larger dispute-path GC witness.

4 Protocol integration: deferred-proof setting

The partial-binding scheme is designed for the deferred-inputs pattern of §2 where the Groth16 proof is generated *after* encryption. We describe the integration with the BitVM2 bridge dispute flow, which is the canonical instantiation.

4.1 Timeline

Let $T_0 < T_1 < T_2 < T_3$ denote the protocol's four observable events on Bitcoin.

Event	Role	Actions
T_0 (peg-in)	Verifier, off-chain	Compute x_S; run $\mathsf{Enc}^\star(vk, x_S, S, D;\ I)$ for each C&C instance; publish commitments to (ct_2, ct_3) and the DSGC transcript on-chain ($\blacksquare_{\text{com}}$).
T_1 (Kickoff)	Prover, on-chain	Broadcast Kickoff tx; opens challenge window. No proof yet. x_D unknown.
$[T_1, T_2]$	Prover, off-chain	Observe watchtower challenges; select x_D (e.g. current Bitcoin tip, watchtower bitmap); build auxiliary proofs (header chain, state chain); run Groth16 prover to produce $\pi = \mathsf{Prove}(vk, (x_S, x_D), w)$.
T_2 (Assert)	Prover, on-chain	Broadcast Assert tx with Lamport reveals of π_1 and x_D; GC gate-by-gate unlocking yields $ct_1 = r \cdot \pi_1$ and $ct_1' = r \cdot P_D + r \cdot B$. Run $\mathsf{Dec}^\star(\pi, (x_S, x_D), ct)$ off-chain to obtain msg.
T_3 (Take)	Prover, on-chain	Spend dispute UTXO via hashlock RIPEMD160(SHA256(msg)). Take1/Take2 scripts enforce byte-equality between Lamport-revealed x_D and the proof's x_D.

4.2 Why partial-binding is required here

At T_0, the Verifier does not know the Groth16 proof or the runtime values x_D. Basic BABE cannot encrypt: its Enc (Alg. 1, line 2) requires $\sum_{j=0}^{n} x_j L_j$, which needs every x_j. Postponing encryption until T_2 is not viable because the Verifier's role ends at peg-in; keeping Verifier online and interactive until dispute time contradicts the trust model (Verifier committee may rotate, become unreachable, or be assumed cryptographically absent after peg-in).

Partial-binding lifts exactly this constraint: $\mathsf{Enc}^\star$ (Alg. 3) needs only x_S, which is peg-in-knowable. The runtime values are absorbed at decryption (Alg. 5, lines 2–5) using the DSGC outputs.

4.3 Consistency across the timeline

Correctness and soundness across the full timeline reduce to four composable properties, each discharged by a distinct mechanism.

Property	Enforced by
x_S is peg-in-fixed	ct_3 binding via Y_S in $\mathsf{Enc}^\star$ (Lemma 2, prefix soundness).
π is valid for (x_S, x_D)	Lemma 5: the DSGC binds the blinded dynamic term $r \cdot P_D + r \cdot B$ to Lamport-committed x_D; a mismatched proof fails bilinear cancellation.
x_D is unique (no equivocation)	Lamport commitment on x_D in Assert tx: double-reveal discloses secret and burns deposit.
x_D used in π matches Lamport-committed x_D	DSGC binding (§3.2, Lemma 5): the dynamic DSGC output is uniquely determined by Lamport-revealed x_D; substitution attempts fail the bilinear cancellation.

The four properties compose multiplicatively in the security reduction (Lemma 2, Lemma 5).

4.4 Failure modes and their resolutions

Failure	Resolution
Prover produces π with wrong x_S	Verifier equation cancellation in $\mathsf{Dec}^\star$ lines 2–4 fails; msg not recovered; dispute path blocked.
Prover chooses x_D not consistent with on-chain observations (e.g. fails watchtower dual-check)	Circuit-internal constraints reject witness; Prove has no output; Prover cannot construct π.
Prover attempts to reveal two distinct x_D values	Lamport equivocation exposes secret key; Verifier slashes deposit.
Prover generates π too late for the challenge window	Protocol liveness failure, not security. Operator liveness parameters must size the window against Groth16 proving time.
Verifier garbles DSGC incorrectly	BABE C&C soundness: the opened instance subset is re-garbled from seed and checked for equality; Prover aborts before deposit if any opened instance disagrees.

4.5 Protocol invariant

Theorem 1 (Deferred-proof correctness)**.** *Under Lemmas 1, 2, 5, together with Lamport one-wayness and cut-and-choose soundness inherited from BABE, the following holds: for every ciphertext* $ct \leftarrow \mathsf{Enc}^\star(vk, x_S, S, D;\ I)$ *produced at* T_0 *and every PPT Prover strategy over* $[T_0, T_3]$*, the Prover obtains* msg *if and only if they commit on-chain to a specific* $x_D^\dagger$ *at* T_2 *and produce a valid Groth16 proof* $\pi^\dagger$ *for* $(x_S, x_D^\dagger)$*.*

Proof sketch. ($\Leftarrow$) is Lemma 1. ($\Rightarrow$) follows by Lemma 5: absent a $\pi^\dagger$ consistent with Lamport-committed $x_D^\dagger$, the bilinear cancellation in $\mathsf{Dec}^\star$ fails. Absent the committed x_S, the Y_S factor mismatches (Lemma 2). □

5 Security analysis

5.1 Main theorem

Lemma 2 (Partial-binding security)**.** *Let $\mathcal{A}$ be a PPT adversary against the partial-binding WE scheme defined by Algorithms 3–5. There exists a PPT reduction such that*

$$\mathrm{Adv}^{\mathrm{pbWE}}_{\mathcal{A}}(\lambda) \;\le\; \mathrm{Adv}^{\mathrm{BABE}}(\lambda) \;+\; \mathrm{Adv}^{\mathrm{DLog}}_{\mathbb{G}_1}(\lambda).$$

Proof sketch. The scheme differs from basic BABE in two places: the single-output scalar-mul GC is replaced with the DSGC of Algorithm 4, and the mask in ct_3 uses Y_S rather than Y. In the final construction this masked term is $\widetilde{Y}_S = Y_S \cdot e(B,\gamma)^{-1}$, where the blinding point B is verifier-chosen and hidden inside the DSGC. GC security bounds the probability that an adversary learns unintended outputs of the DSGC evaluation. The Y-to-$\widetilde{Y}_S$ shift is reconciled via the bilinearity identities (3) and (4) inside the decryption equation. Any attack that recovers msg without a valid proof extracts proof elements from the GC transcript, yielding an attack on basic BABE. □

5.2 r-leakage analysis

Lemma 3 (No new r-leakage)**.** *Let $\mathcal{B}$ be a PPT adversary given the artifacts published by* $\mathsf{Enc}^\star$ *plus the dispute-time DSGC outputs (ct_1, ct_1'). Then*

$$\mathrm{Adv}^{\textit{r-leak}}_{\mathcal{B}}(\lambda) \;\le\; \mathrm{Adv}^{\mathrm{DLog}}_{\mathbb{G}_1}(\lambda) \;+\; \mathrm{Adv}^{\mathrm{coDLog}}(\lambda).$$

Proof sketch. Public exposures of r-scaled samples are $ct_2 = r\cdot[\delta]_2$, $ct_1 = r\cdot\pi_1$, and $ct_1' = r\cdot P_D + r\cdot B$. These are multi-instance DLog challenges with a shared scalar; self-reducibility reduces to single-instance DLog. Cross-group combinations add at most a co-DLog term. In particular, the blinding term prevents the one-dimensional GOAT setting from leaking $r \cdot L_D$ by direct division through a known scalar x_D. □

5.3 x_D-substitution attack on a naive scheme

A naïve instantiation of partial-binding WE that publishes $ct_D = \{r \cdot L_j\}_{j\in D}$ and lets the decryptor form $r \cdot P_D$ off-chain from x_D and ct_D admits the following attack:

Lemma 4 (Substitution attack on naive ct_D scheme)**.** *Let $\mathcal{A}$ control a Prover possessing a valid proof π' for (x_S, x'_D), and let $x^\dagger_D \neq x'_D$ be the value that enclosing-protocol commitments (Lamport reveal, chain-derived bitmap) require. Against the naive ct_D instantiation, $\mathcal{A}$ recovers* msg *via* $\mathsf{Dec}^\star$ *by (i) Lamport-committing $x^\dagger_D$ on-chain and (ii) using x'_D off-chain to form $r \cdot P_{D'}$. No on-chain check discriminates between $x^\dagger_D$ (committed) and x'_D (used in decryption) because the proof's public-input vector is not script-verifiable.*

Proof sketch. In the naive scheme, $r \cdot P_D$ is Prover-computed from public ct_D and arbitrary x_D; the verifier equation cancellation succeeds for (x_S, x'_D, π'). Lamport on $x^\dagger_D$ pins *that* value for on-chain equality checks but provides no link to the off-chain computation of $r \cdot P_D$. □

5.4 DSGC binding closes the attack

The construction of §3 replaces the naive plaintext ct_D interface with the DSGC of Algorithm 4: the dynamic term becomes the blinded GC output $r \cdot P_D + r \cdot B$, parameterized by Lamport-committed x_D, rather than a Prover-chosen off-chain quantity.

Lemma 5 (DSGC binding)**.** *Let $\mathcal{A}$ control a Prover who produces* msg *via* $\mathsf{Dec}^\star$ *instantiated with Algorithm 4. Let x_D^{Lam} denote the Lamport-revealed dynamic value and x_D^{π} the public-input vector of $\mathcal{A}$'s proof. Then $x_D^{\mathrm{Lam}} = x_D^{\pi}$ except with probability*

$$\mathrm{Adv}^{\mathrm{BABE}}(\lambda) + \mathrm{Adv}^{\text{GC-sec}}(\lambda) + \mathrm{Adv}^{\mathrm{Lamport}}(\lambda) + \mathrm{Adv}^{\text{G16-KS}}(\lambda).$$

Proof sketch. Two facts, each reducing to a named assumption:

(a) GC privacy hides $\{r \cdot L_j\}_{j \in D}$ and $r \cdot B$. Constants baked into the DSGC at garbling time are not extractable by the evaluator: by GC privacy, the evaluator's view on a single evaluation reveals only the output labels for the specific inputs provided, not constants or intermediate wire values (Lindell–Pinkas; Bellare–Hoang–Rogaway). Hence the adversary cannot directly compute $r \cdot P_{D'}$ for $x'_D \neq x_D^{\mathrm{Lam}}$ by combining extracted $\{r \cdot L_j\}$, nor can it peel off the blinding term.

(b) Lamport soldering fixes DSGC input to x_D^{Lam}. By construction, the DSGC input-wire labels for x_D are derived from Lamport preimages ($L^b_{j,i} = \mathrm{KDF}(s^b_{j,i})$). To evaluate the DSGC with some x'_D, the Prover needs labels $\{L^{b'}_{j,i}\}$ for x'_D's bit pattern. Revealing preimages for a bit pattern different from the on-chain Lamport reveal constitutes equivocation: it simultaneously exposes both halves of at least one Lamport slot, which is slashable.

Combining (a) and (b): any DSGC evaluation accessible to the Prover yields $r \cdot P_D + r \cdot B$ with $x_D = x_D^{\mathrm{Lam}}$. Therefore line 2 of Alg. 5 computes $Q_{\mathrm{blind}} = e(P_{D,\mathrm{Lam}}, \gamma)^r \cdot e(B, \gamma)^r$. The decryption quotient $r \cdot Y / Q_{\mathrm{blind}}$ evaluates to $r \cdot \widetilde{Y}_S$ iff $x_D^{\pi} = x_D^{\mathrm{Lam}}$; otherwise the residual factor $e(P_{D,\pi} - P_{D,\mathrm{Lam}}, \gamma)^r \neq 1$ and the mask $H(r \cdot \widetilde{Y}_S)$ is not recovered. A Prover who still outputs msg either inverts H or produces a proof extracting to no witness, contradicting Groth16 KS. □

Remark 2 (Proactive vs. reactive binding)**.** *The DSGC mechanism binds x_D proactively through the garbled circuit's structure: substitution is infeasible without breaking GC privacy or Lamport one-wayness, and no Challenger is required. In contrast, a disprove-based design would rely on an active Challenger running the full Groth16 verifier and posting a Disprove transaction upon detecting $x_D^{\pi} \neq x_D^{\mathrm{Lam}}$, reintroducing full-verifier cost on the dispute path. DSGC preserves BABE's core "no full verifier on chain" property.*

Remark 3 (Both GC privacy and soldering are required)**.** *Lemma 5 depends on* both *assumptions jointly. Weakening either enables the attack: if the Verifier published $\{r \cdot L_j\}$ in plaintext (violating GC privacy) the Prover forms $r \cdot P_{D'}$ directly; if the DSGC input wires for x_D are not soldered to Lamport preimages (allowing public label derivation) the Prover evaluates the DSGC on any x'_D of their choice. The construction therefore requires that $\{r \cdot L_j\}$ be baked as garbled constants* and *that x_D input wires be properly soldered. Both are standard ingredients of BABE's GC infrastructure applied to the extended input space.*

6 Cost

Item	Basic BABE	Partial-binding (delta)
GC: output count	$r \cdot \pi_1$	$+\left(r \cdot P_D + r \cdot B\right)$ (DSGC)
GC: blinded offset	none	$+\, r \cdot B$
GC: gate count	$\sim 4.7 \times 10^8$	$+\ \sim \lvert D\rvert \cdot 1.5 \times 10^8$ (fixed-base)
Encryption: $\mathbb{G}_1$ scalar-muls	$O(n)$	$+\lvert D\rvert$ (baked into DSGC constants)
Encryption: pairings	2	unchanged
Ciphertext size	baseline	unchanged
Decryption: $\mathbb{G}_1$ scalar-muls	–	$+\lvert D\rvert$ (inside DSGC evaluation)
Decryption: pairings	2	$+1$
Decryption: $\mathbb{G}_T$ divisions	1	$+1$
Lamport slots on-chain	π_1 only (˜508 bits)	$+\lvert D\rvert \cdot 254$ bits for x_D
On-chain primitives	hashlock, GC, Lamport	unchanged; witness size increases on dispute path

For the BitVM2 bridge with $|D| = 4$ field slots: DSGC grows to $\sim 1.1 \times 10^9$ gates (roughly 2.3× baseline), dispute-path Taproot-unlock witness grows proportionally, and the Lamport witness at Assert grows by $\sim 10^3$ bits. The happy path remains cheap: the Prover evaluates DSGC off-chain to recover msg and spends via hashlock. Full on-chain GC unlocking runs only in the rare dispute path.

7 Conclusion

Partial-binding WE resolves the full-public-input requirement of basic BABE for protocols with deferred public inputs. The construction uses a Dual-Scalar Garbled Circuit (§3.2) whose two outputs—$r \cdot \pi_1$ and $r \cdot P_D + r \cdot B$—together provide prefix binding on x_S and proactive cryptographic binding on x_D without leaking low-dimensional basis terms. The reduction stays within standard BABE assumptions plus GC-security and Lamport one-wayness. No new on-chain primitive is introduced; BABE's core property of avoiding a full on-chain Groth16 verifier is preserved.

A Universal-Composability analysis

We sketch a UC treatment of the partial-binding WE scheme. A full simulator-based proof is deferred; this appendix fixes the functionality, the hybrid model, and the simulation outline sufficient for a reviewer to verify the security architecture.

A.1 Ideal functionality $\mathcal{F}_{\text{pbWE}}$

$\mathcal{F}_{\text{pbWE}}$ interacts with Verifier V, Prover P, and adversary $\mathcal{S}$. Internal state is a per-session record indexed by sid.

Setup (at T_0): Upon $(\mathsf{ENC}, \mathsf{sid}, vk, x_S, S, D, \text{msg})$ from V:
store $(\mathsf{sid}, vk, x_S, S, D, \text{msg}, \perp)$; send $(\mathsf{ENCRYPTED}, \mathsf{sid}, x_S, S, D)$ to $\mathcal{S}$ and all parties. msg *is not leaked to* $\mathcal{S}$.

Commit (at T_2): Upon $(\mathsf{COMMIT}, \mathsf{sid}, x_D, h_{\pi_1})$ from P:
if previously committed, reject (equivocation); else store $\mathsf{committed} := (x_D, h_{\pi_1})$ and send $(\mathsf{COMMITTED}, \mathsf{sid}, x_D, h_{\pi_1})$ to $\mathcal{S}$.

Decrypt (at T_2): Upon $(\mathsf{DEC}, \mathsf{sid}, \pi)$ from P:
require prior commit (x_D^*, h^*); return $(\mathsf{DEC\_OK}, \mathsf{sid}, \text{msg})$
iff $\mathsf{Verify}_{\mathrm{G16}}(vk, (x_S, x_D^*), \pi) = 1$ and $H(\pi.\pi_1) = h^*$; else $(\mathsf{DEC\_FAIL}, \mathsf{sid})$.

The functionality encodes four security properties by construction: prefix binding (on x_S), proactive x_D binding (at COMMIT), decryption conditional on valid proof matching the committed (x_S, x_D), and equivocation prevention.

A.2 Hybrid model and trust setup

The real protocol is analyzed in the $(\mathcal{F}_{\mathrm{CRS}}, \mathcal{F}_{\mathrm{BTC}}, \mathcal{F}_{\mathrm{RO}})$-hybrid model:

- $\mathcal{F}_{\mathrm{CRS}}$: ideal Groth16 trusted setup delivering vk.
- $\mathcal{F}_{\mathrm{BTC}}$: idealized Bitcoin ledger exposing transaction inclusion, Taproot unlock, and Lamport preimage reveal.
- $\mathcal{F}_{\mathrm{RO}}$: random oracle modeling H.

A.3 Simulator outline

The simulator $\mathcal{S}$ plays the real-world adversary's role against $\mathcal{F}_{\mathrm{pbWE}}$:

- *At ENC.* $\mathcal{S}$ learns (x_S, S, D) but not msg. Samples $\widetilde{\text{msg}}$ uniformly and runs $\mathsf{Enc}^\star$ honestly with $\widetilde{\text{msg}}$, producing a simulated ciphertext $\widetilde{ct}$. Indistinguishability reduces to partial-binding security (Lemma 2).
- *At COMMIT.* $\mathcal{S}$ extracts (x_D, π_1) from the Prover's Lamport reveals (Lamport one-wayness gives a unique extraction). Forwards $(x_D, H(\pi_1))$ to $\mathcal{F}_{\mathrm{pbWE}}$.
- *At DEC.* $\mathcal{S}$ applies the Groth16 KS extractor to π, forwards π to $\mathcal{F}_{\mathrm{pbWE}}$. On $(\mathsf{DEC\_OK}, \text{msg})$, $\mathcal{S}$ programs $\mathcal{F}_{\mathrm{RO}}$ so that $\widetilde{ct}_3 \oplus H(r \cdot \widetilde{Y}_S) = \text{msg}$, preserving transcript consistency. On $(\mathsf{DEC\_FAIL})$, $\mathcal{S}$ emits the residual garbage that $\mathsf{Dec}^\star$ would compute (Lemma 5).

A.4 Hybrid argument

The real-ideal indistinguishability is established by a chain of hybrids, each justified by a standard assumption:

Hybrid step	Justification
Real π_{pbWE}	—
ct_3 masked by RO output	programmable $\mathcal{F}_{\mathrm{RO}}$
DSGC transcript simulated with random labels	GC-security of Alg. 4
Lamport reveals simulated via trapdoor	Lamport one-wayness
Groth16 π extracted	Groth16 KS in AGM
$\widetilde{\mathsf{msg}}$ replaced with real msg	$\mathcal{F}_{\mathrm{pbWE}}$ on $\mathsf{DEC\_OK}$
Ideal $\mathcal{F}_{\mathrm{pbWE}}$	—

Each step's distinguishing advantage is bounded by a negligible term in λ.

A.5 UC statement (informal)

Theorem 2 (UC emulation)**.** *Under the cryptographic assumptions of §5 and the hybrid model above, there exists a PPT simulator $\mathcal{S}$ such that for every PPT environment $\mathcal{Z}$ and PPT real-world adversary $\mathcal{A}$,*

$$\left|\Pr[\mathsf{EXEC}^{\pi_{\mathrm{pbWE}}}_{\mathcal{A},\mathcal{Z}} = 1] - \Pr[\mathsf{EXEC}^{\mathcal{F}_{\mathrm{pbWE}}}_{\mathcal{S},\mathcal{Z}} = 1]\right| \;\leq\; \mathrm{negl}(\lambda).$$

A.6 Composition consequences

UC emulation yields:

- *Sequential composition.* Multiple peg-in sessions (distinct sid) remain secure simultaneously.
- *Subroutine replacement.* Replacing Groth16 by any UC-secure SNARK preserves $\mathcal{F}_{\mathrm{pbWE}}$ emulation, modulo the replacement's own UC proof.
- *Enclosing-protocol compatibility.* Partial-binding WE composes with adaptor signatures, Taproot disprove flows, and higher-layer bridge protocols without rerunning its security proof.

A full mechanized UC proof is future work; §5 gives the game-based analogue used for concrete parameters.

References

[1] Goat Research Team. BABE: Verifying Proofs on Bitcoin Made 1000× Cheaper. IACR ePrint 2026/065.

[2] J. Groth. On the Size of Pairing-Based Non-interactive Arguments. EUROCRYPT 2016, pp. 305–326.

[3] R. Canetti. Universally Composable Security: A New Paradigm for Cryptographic Protocols. FOCS 2001, pp. 136–145; updated in IACR ePrint 2000/067.